

Updated script is at `artifacts/RcloneAutomation.sh`. The GPG blob and both live presets are unchanged.

### Bugs that would have hurt you

- After decrypt, the loop **printed the rclone password, salt, and both AWS keys** to the terminal. That looked like leftover debug.
- Credentials were exported as `RCLONE_CONFIG_AWSDEEP_*`, but the command uses an on-the-fly `:s3:... remote with --config=/dev/null. Those named-remote vars never applied, so rclone would only work if something else on the machine already had AWS creds.`
- `set -e plus "${cmd[@]}"; rc=$?` meant a failed rclone never reached `unset`, and the names being `unset ( s3_access_key_id / s3_secret_access_key )` were never set in the first place.
- Banner said “sync”; the command is `rclone copy` (does not delete extras at the destination). I left `copy` — that is the safer default for Glacier.

### What I changed

- Secrets stay in the environment only. An `EXIT / INT / TERM` trap wipes them. `HISTFILE` is unset.
- Same AWS keys are also exported as `RCLONE_S3_*` and `AWS_*` so `:s3:glacier-deep-archive-backup-ab/...` actually authenticates.
- `gpg / rclone` presence, non-empty decrypt, required key names, and source path are checked before launch.
- Log name no longer embeds `:s3:` colons. `dtach` sockets go under `$XDG_RUNTIME_DIR/rclone-dtach` (or `/tmp`). If `dtach` is missing, it runs in the foreground instead of dying.
- `--bwlimit-file` became `--bwlimit` so `BW=250M` is a session cap, not a per-file cap.
- `Crypt remote secret` is prepared with `rclone obscure on password + salt`. `RCLONE_CONFIG_SECRET_REMOTE` is still commented — your active jobs are plaintext Deep Archive. Uncomment that and point a preset at `secret:` if you want client-side encryption.

Type `yes` after the command preview, same as before. Reattach a `dtach` session with the path the script prints